

PARTE 5 - QUESTÕES SITUACIONAIS

Teórica - Avaliação Comportamental e Senioridade

Responda as questões abaixo de forma objetiva e com profundidade técnica. Não há resposta única correta - o avaliador buscara consistência de raciocínio, maturidade profissional e domínio da área.

Questão 1 - Requisito Ambíguo

Durante o refinamento, você identifica que no requisito "o sistema deve validar o livro antes de salvar" está completamente vago. O PO está indisponível por 2 dias e o desenvolvimento já começou assumindo uma interpretação. Qual é sua postura? Como você garante que os testes serão relevantes sem ter clareza total do comportamento esperado? Quais riscos você documenta?

Sua resposta:

Procurou buscar reduzir os riscos imediatamente (não espera o PO voltar, registrar a dúvida de forma clara e visível e criar testes que não dependem de uma única interpretação) e registrar claramente a incerteza.

Primeiro conversei com o time de desenvolvimento para entender exatamente qual regra eles assumiram, mas para registrar o comportamento atual do sistema que já está sendo construído.

Depois eu documentei no refinamento ou na documentação de testes algo como: "Não está claro o que significa validar o livro. Pode ser validação de campos, duplicidade, formato ou regras específicas."

Questão 2 - Pressão por Release

Faltam 4 horas para o deploy em produção de uma feature crítica. Você identificou 2 bugs classificados como "Alto": um bug funcional que afeta 30% dos cenários de busca e um bug de schema que expõe dados parcialmente incorretos. O Product Owner pressiona para liberar, argumentando que "nenhum usuário real vai perceber isso agora". Qual é sua posição? Como você conduz essa conversa? O que você documenta?

Sua resposta:

Sendo o deploy de uma feature crítica eu como QA analisando o impacto real e o risco futuro, não recomendo o deploy com bugs classificados como alto impacto. Tento explicar de forma simples o impacto: o bug afeta 30% das buscas. Isso significa que uma parte dos usuários pode não encontrar o que precisa ou ter resultados errados.

Já o bug do schema faz o sistema retornar dados errados. Isso pode gerar confusão e perda de confiança no sistema.

Depois trago um ponto principal: mesmo não sendo visível para todos agora, ele já está afetando o comportamento correto do sistema.

Ofereço alternativas para não travar o negócio: corrigir pelo menos o bug do schema antes do deploy, adiar por algumas horas para correção rápida e como último caso liberar com rollback pronto caso algo pior aconteça.

Documento:

- Descrição dos dois bugs

- Severidade

- Impacto que irá causar: 30% dos cenários de busca afetados e dados retornados incorretamente no schema

- Riscos em produção: perda de confiabilidade, comportamento inconsistente e suporte elevado após release

- Recomendação técnica: não liberar em produção

- Decisão final do PO

- Estratégia de mitigação (rollback, monitoramento)

Questão 3 - Mentoria de Júnior

Um colega testador júnior está responsável por testar um endpoint complexo (PATCH com regras de negócio elaboradas). Após 2 dias, ele entrega apenas 5 casos básicos, todos passando, sem nenhum caso negativo ou de borda - e está empolgado com o resultado. Como você aborda essa situação? Como você o mentoreia sem desmotivar? Que exercício prático você proporia para desenvolver o pensamento analítico dele?

Sua resposta:

Primeiro eu começo validando o trabalho: deixo claro que ele conseguiu validar o fluxo básico e que isso é importante porque garante a funcionalidade principal.

Depois de forma simples eu falo: como esse endpoint tem regras de negócio complexas, só cenários positivos não são suficientes para garantir a qualidade.

Ao invés de dizer que ele errou, direcionar ele para reflexões do tipo:

“O que acontece se alguém enviar dado vazio?

E se envia valor fora do limite?

E se tenta atualizar algo que não pode ser alterado?”

Como ele já fez o fluxo principal eu tento direcioná-lo para pensar em ricos, explicando que além de testarmos para ver se funciona, também precisamos quebrar o sistema de forma controlada, deixando claro que ele está no caminho certo, mas precisa expandir o olhar.

Eu peço que ele faça algo bem estruturado como:

Pegar um mesmo endpoint e montar 3 camadas sendo elas:

Cenários positivos (o que ele já havia feito)

Cenários negativos

Casos de borda

Questão 4 - Falsos Positivos na Automação

A suite de automação de regressão no pipeline de CI/CD apresenta 15% de taxa de falsos positivos (testes que falham sem bug real). Isso gerou desconfiança no time e alguns desenvolvedores começaram a ignorar falhas no pipeline. O que você faz imediatamente? Como você investiga e trata os falsos positivos? Como você recupera a credibilidade da suite?

Sua resposta:

Minha ação imediata seria restaurar a confiança no pipeline. Eu identificaria os testes responsáveis pelos falsos positivos, analisaria suas causas raízes (sincronização, dados, dependências ou ambiente) e corrigiria ou isolaria temporariamente os testes instáveis. Em seguida, monitoraria métricas de estabilidade e apresentaria os resultados ao time. O objetivo é que uma falha no pipeline volte a representar um problema real, recuperando a credibilidade da suite de automação e evitando que os desenvolvedores ignorem alertas importantes.

Questão 5 - Cobertura Insuficiente em Área Crítica

As métricas mostram que o módulo de autenticação/autorização tem apenas 12% de cobertura automatizada, enquanto o catálogo de livros tem 78%. A sprint está na metade e o backlog está cheio. Como você prioriza? Qual argumento você usa para o time e para o PO para justificar o investimento em cobertura de autenticação? Como equilibra velocidade de entrega com qualidade?

Sua resposta:

Eu priorizaria pelo risco e não apenas pela cobertura. Embora o catálogo tenha cobertura maior, autenticação e autorização são módulos críticos porque controlam acesso ao sistema. Eu justificaria para o time e para o PO que uma falha nessa área tem impacto muito maior para o negócio do que uma falha em funcionalidades já bem cobertas. Para equilibrar velocidade e qualidade, eu focaria na automação dos fluxos mais críticos de autenticação, aumentando a proteção do sistema sem comprometer significativamente as entregas da sprint.

Questão 6 - Conflito Dev x QA sobre Severidade

Você registrou um bug como "Crítico": ao deletar um livro com capa associada, a capa permanece como registro órfão no banco (sem FK válida). O desenvolvedor argumenta que é "cosmético" e deveria ser "Baixo", pois "não aparece para o usuário". A discussão está ficando acalorada na reunião. Como você fundamenta sua avaliação de severidade? Como você conduz a mediação? Quais critérios objetivos você utiliza para classificar severidade de bugs?

Sua resposta:

Eu evitaria discutir com base em opiniões e conduziria a conversa usando critérios objetivos de severidade definidos pelo time. O fato de o problema não ser visível para o usuário não significa que ele seja baixo impacto, pois há comprometimento da integridade dos dados. Porém, antes de manter a classificação como crítica, eu avaliaria fatores como perda de dados, indisponibilidade, impacto operacional e risco futuro. Minha função é apresentar os impactos técnicos e de negócio para que a severidade seja definida com base em critérios consistentes e não em percepções individuais.